TITLE

Breast Cancer Prediction Using Machine Learning Models

23 June 2023

TEAM MEMBERS DETAILS

The table below shows the task assignment for team members along with their estimated time as the follow workflow

No.	Date	Tasks	Name / Email
1	**28 Jun**	Data Specification & Pre-processing	Asrar Qaseem asrarqassem@my.unt.edu
2	**1 July**	Data Analysis and Integration	
3	**8 July**	Model Development: Support Vector Classification Model	
4	**15 July**	Model Development: K-Nearest Neighbors (KNN) Model	
5	**17 July**	Evaluation and Comparison	
6	**18 July**	Graphical User Interface and Web Development	

PROJECT ABSTRACT

Breast cancer is a significant health challenge, being the most common cancer in women and a major cause of cancer-related deaths. The early detection of breast cancer plays a crucial role in improving patient outcomes. Therefore, this project aims to utilize machine learning models to enhance breast cancer detection and diagnosis.

First of all, the advancements in technology and machine learning have revolutionized the field of cancer detection. Machine learning techniques, such as classification algorithms and deep learning architectures, have shown promising results in identifying patterns and characteristics indicative of breast cancer. By leveraging these models, we can analyze large and diverse datasets comprising clinical, genetic, and imaging data to develop accurate and reliable diagnostic tools.

Anyhow, the other objective of this project is to train machine learning models using comprehensive datasets, such as the Wisconsin Breast Cancer Dataset (WBCD), to accurately classify breast cancer cases. These models will learn from the data and identify patterns and features that distinguish between benign and malignant tumors. Thus, the goal is to create robust and efficient models that can aid healthcare professionals in making informed decisions and improving the accuracy and efficiency of breast cancer diagnosis.

By leveraging machine learning models in breast cancer detection, we can potentially revolutionize current diagnostic practices. Early detection allows for timely interventions and personalized treatment strategies, which can significantly improve patient outcomes. Furthermore, these models have the potential to optimize resource allocation in healthcare systems, improve efficiency, and contribute to research insights.

In addition, this project aims to bridge the gap between cutting-edge machine learning models and the healthcare sector. By integrating these models into the healthcare system, healthcare professionals can benefit from the advancements in technology and improve their diagnostic capabilities. This transformative approach has the potential to empower healthcare professionals, facilitate early detection, and ultimately improve the prognosis for patients with breast cancer.

In summary, this project focuses on utilizing machine learning models to enhance breast cancer detection and diagnosis. By leveraging advanced techniques and comprehensive datasets, the goal is to develop accurate and efficient models that can aid in early detection, personalized treatment planning, and improved patient outcomes. The integration of these models into the healthcare sector has the potential to revolutionize breast cancer diagnosis, optimize resource allocation, and contribute to research insights.

Visual Flow Diagram

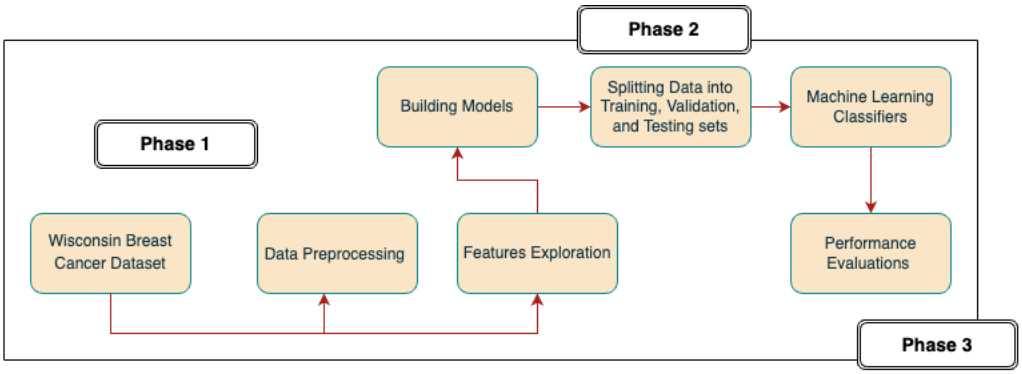


Figure 1: Workflow Diagram for Breast Cancer Detection Using Machine Learning Models

As shown in Fig1, which I built to show how Machine Learning is used to find breast cancer. This figure provides how the model works and summarizes the step-by-step process for this project. In addition, from this figure, we can get an idea of how the machine learning models can work to understand the data and how these models can detect breast cancer cells and provide the results with high accuracy. In order to effectively handle the models, I planned to divide the project into three phases, each of which would be responsible for completing its own responsibilities. This diagram visually shows the sequential stages, starting with data collection and preprocessing, followed by feature selection and engineering. It then shows the model training and evaluation phase, followed by the prediction generation and performance monitoring steps. Hence, it will be useful to demonstrate a valuable perspective for understanding the entire process of Breast Cancer detection by using Machine Learning models, allowing users to grasp the workflow and gain insights into the underlying mechanisms of the machine learning model in this field.

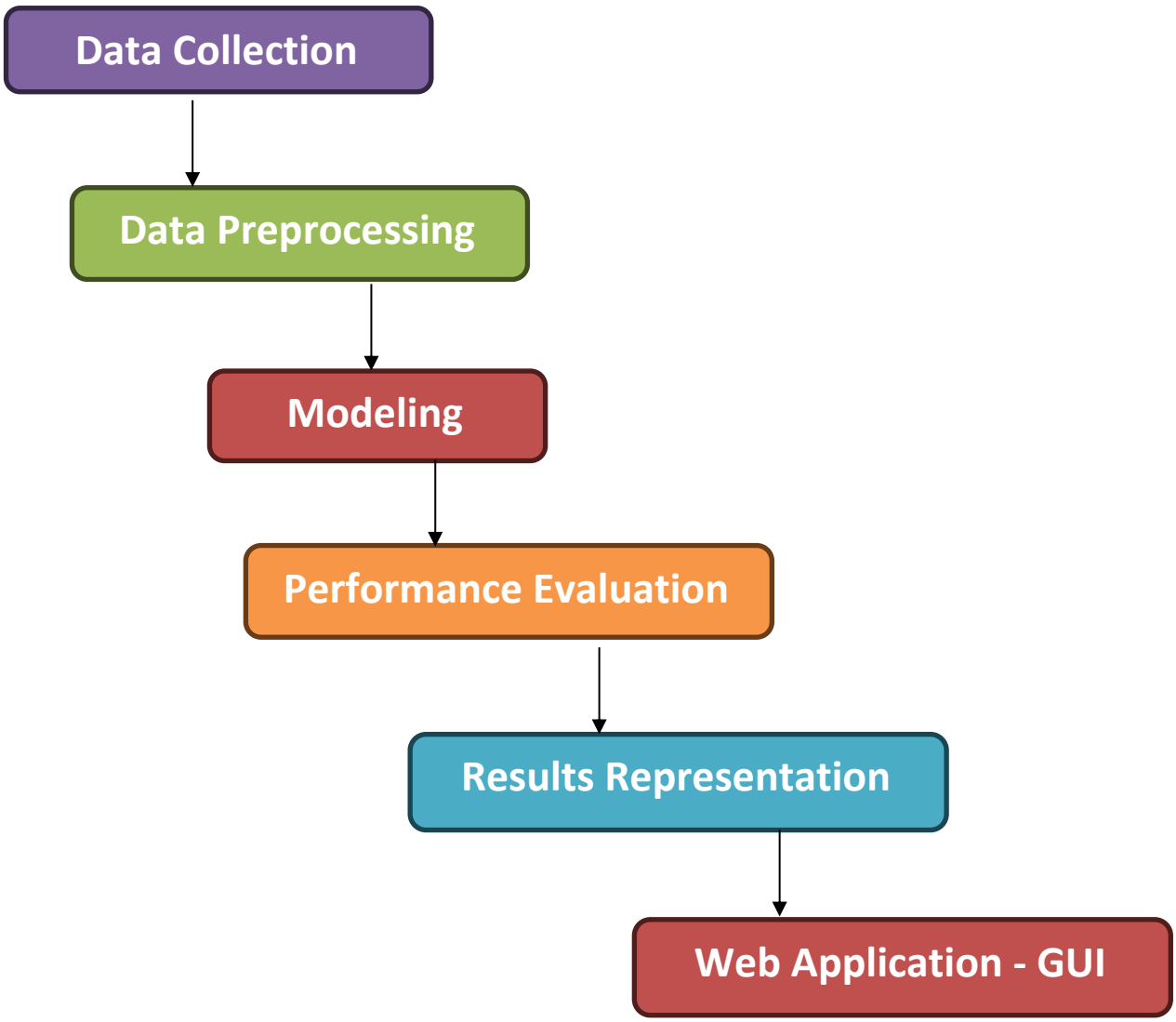
The first phase of the project will mainly focus on data collection, data cleaning, preprocessing, and other necessary operations. This phase plays a crucial role in obtaining accurate and reliable results, which, in turn, are essential for creating a high-performing model. Data collection involves identifying and retrieving the required datasets from reliable sources, such as the Wisconsin Breast Cancer Dataset [WBCD], while data cleaning and preprocessing involve, addressing the significant data, and transforming the data into a suitable format for analysis. These operations lay the foundation for achieving a robust and accurate model by ensuring the integrity and quality of the data used in subsequent phases of the project.

The second phase of the project is to construct the model. This phase entails the selection of suitable machine learning models, such as the Support Vector Classification Model, and the K-Nearest Neighbors (KNN) Model which will be trained using the preprocessed data. The aim of this phase is to create a model that utilizes the identified features and traces the target, which is diagnosis, to effectively detect if cells are benign or malignant. The objective is to design a robust and accurate model that can analyze the provided data and make precise predictions about the presence of cancer cells. By harnessing the power of machine learning techniques, the goal is to enhance the ability to detect cancer at an early stage and improve the overall accuracy of cancer

identification. The project seeks to leverage the identified features and target variables to develop a reliable and efficient model that can contribute to improved cancer detection and diagnosis.
During the final phase of the project, particular attention will be given to testing and validating the model that has been developed. This crucial stage entails assessing the performance and accuracy of the model in effectively detecting breast cancer using real data. Strong evaluation techniques will be employed to measure the model's ability to accurately identify cancerous cells and provide reliable results. In short, by thoroughly evaluating the model's performance, we can gain valuable insights into its effectiveness and make any necessary refinements to optimize its detection capabilities for breast cancer.

Project Framework

This project framework provides a structured approach and set of guidelines for organizing and executing complex projects, ensuring consistency and efficiency throughout the building and development process of the model.



Data specification

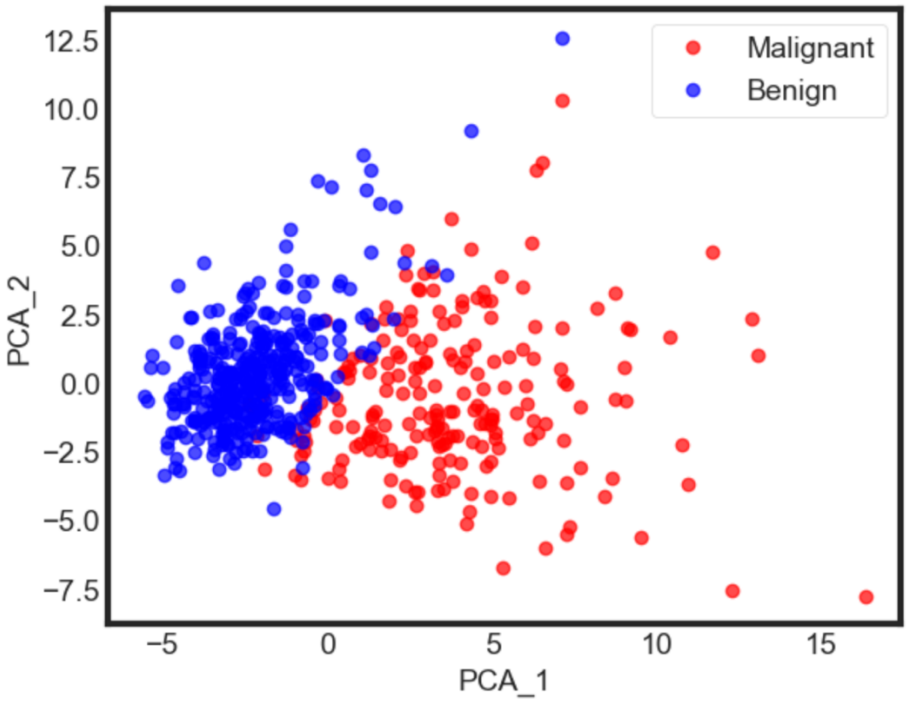
Dataset Description: The WBCD (Wisconsin Breast Cancer Diagnostic) dataset is commonly used in breast cancer detection projects. It comprises samples obtained from patients with suspected breast cancer. Each sample is associated with various features extracted from digitized images of fine needle aspirate (FNA) of breast mass. The goal is to develop machine learning algorithms that can accurately classify tumors as either benign or malignant based on these features.
Data Size: The WBCD dataset contains a total of 569 records or instances, where each instance represents a different patient. Each record is characterized by 32 columns, comprising both input features and the target variable (diagnosis).
Input Features: The dataset includes 30 real-value input features (columns 3-32) derived from the digitized images of FNA. These features capture different characteristics of the cells, such as radius, texture, perimeter, area, smoothness, compactness, concavity, symmetry, fractal dimension, and more. These features serve as inputs to the machine learning algorithms for training and prediction purposes.

```
[122]: Index(['diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
               'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
               'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
               'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
               'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
               'fractal_dimension_se', 'radius_worst', 'texture_worst',
               'perimeter_worst', 'area_worst', 'smoothness_worst',
               'compactness_worst', 'concavity_worst', 'concave points_worst',
               'symmetry_worst', 'fractal_dimension_worst'],
              dtype='object')
```

- Radius: The radius feature represents the average distance from the center to points on the perimeter of the cell nucleus. It provides information about the size of the nucleus.
- Texture: The texture feature relates to the variation in grayscale intensities of the pixels within the cell nucleus. It measures the standard deviation of gray-level values, which can indicate variations in cell composition.
- Perimeter: The perimeter feature denotes the length of the boundary of the cell nucleus. It can provide insights into the irregularity or smoothness of the cell shape.
- Area: The area feature represents the total area occupied by the cell nucleus. It can provide information about the size of the nucleus and potentially correlate with the overall cell growth.
- Smoothness: The smoothness feature characterizes the local variations in radius lengths of the cell nucleus. It quantifies the average difference between the radius lengths of adjacent points on the nucleus perimeter. Higher smoothness values indicate a more regular and uniform cell shape.
- Compactness: The compactness feature measures the shape irregularity of the cell nucleus. It is calculated as the ratio of the perimeter squared to the area. Higher compactness values suggest a denser and more irregularly shaped nucleus.

- Concavity: The concavity feature represents the severity of concave portions of the cell nucleus. It quantifies the number and depth of concave regions within the boundary of the nucleus.
- Symmetry: The symmetry feature reflects the symmetry of the cell nucleus. It measures the similarity between the radial distances of points on opposite sides of the nucleus. Higher symmetry values indicate a more symmetrical shape.
- Fractal Dimension: The fractal dimension feature characterizes the complexity of the cell nucleus boundary. It quantifies the irregularity and complexity of the perimeter using fractal geometry concepts.

Target: The target variable in the WBCD dataset is the diagnosis of the tumor. It indicates whether the tumor is "benign" or "malignant." The diagnosis is provided in the second column of the dataset, with "B" representing benign and "M" representing malignant. This target variable is used to train the ML algorithms to learn the patterns and characteristics associated with each tumor type.



Data Format: The WBCD dataset is typically available in a tabular format, such as a CSV (Comma-Separated Values) file. Each row represents a distinct patient sample, while each column represents a specific feature or attribute.

Data Preprocessing: Prior to training ML algorithms, it is important to preprocess the dataset. This involves handling missing values, dealing with outliers, and performing data normalization or scaling. Additionally, if any categorical variables are present, they may need to be encoded into numerical representations suitable for ML algorithms.

For Splitting the Dataset: To evaluate the performance of ML algorithms, it is common practice to split the WBCD dataset into training and testing subsets. The training set is used to train the models, while the testing set is used to assess the models' performance on unseen data. The recommended split ratio is typically around 80% for training and 20% for testing.

Cross-Validation: in order to ensure the robustness and generalizability of the models, cross-validation techniques can be employed. One commonly used approach is k-fold cross-validation, which involves dividing the data into multiple subsets or folds and iteratively performing training and evaluation across these folds. In detail, when dealing with classification tasks and cross-validation, it is crucial to split the data into training and test sets to prevent overfitting and enable the generalization of the model to unseen data. Cross-validation takes this concept further by dividing the data into equally sized folds or subsets. The training process starts by using all but one-fold as the training data, referred to as the holdout sample. The model is trained on this training set, and its performance is then evaluated using the holdout sample. Subsequently, the holdout sample is reintroduced to the remaining folds, and a different fold is chosen as the new holdout sample. This process of training and evaluation is repeated until each fold has had the opportunity to be a test or holdout sample. The expected performance of the classifier, known as the cross-validation error, is determined by calculating the average error rates obtained from each holdout sample. To demonstrate this process, a standard train-test split is initially performed, and then the cross-validation error is computed. By employing cross-validation, we can obtain a more reliable estimate of the model's performance, ensuring its effectiveness on diverse subsets of the data and enhancing its ability to generalize to new, unseen instances.

```
LR: 0.944808 (0.031145)
LDA: 0.954744 (0.018784)
KNN: 0.937179 (0.028218)
CART: 0.942244 (0.027435)
NB: 0.937115 (0.040822)
SVM: 0.901987 (0.047020)

→ 10-Fold cross-validation accurcay score for the training data for six classifiers
```

Class Imbalance: It is essential to consider any class imbalance in the WBCD dataset. Given the nature of breast cancer prevalence, the number of benign cases may outweigh the number of malignant cases. Handling class imbalance can involve techniques such as oversampling the minority class, under sampling the majority class, or utilizing algorithms that are robust to imbalanced datasets.

Project Design

Describe tools and frameworks:

1- The project relies on Jupyter Notebooks as the primary programming environment for development and analysis, with Python as the chosen programming language. Python's rich ecosystem of libraries and frameworks, including Pandas, NumPy, TensorFlow, Keras, and Scikit-learn, supports data manipulation, statistical analysis, and machine learning algorithms.

2- Prerequired libraries: I used these important libraries, such as NumPy, Pandas, Scikit-learn, and others important libraries.

· NumPy which is a Python library for numerical computing, enabling efficient manipulation of large arrays and matrices, and providing mathematical functions for scientific and data analysis tasks.

· Pandas is a Python library used for data manipulation, analysis, and cleaning. It provides data structures and functions for working with structured data, such as tables and data frames.

· Scikit-learn provides a comprehensive collection of evaluation metrics for assessing the performance of machine learning models across various tasks such as classification, regression, and clustering.

3- To install these important libraries, you can use pip packages which are used as following this command → pip install [Name of libraries]. In addition, these tools and frameworks provide a range of options for conducting the main functions of my project, from data manipulation and visualization to machine learning and advance learning techniques.

Describe Models, List of design of decisions important methods, and functionality of the program as seen by the user:

→ The following list is the most important methods that are used to complete the project:

- Download required libraries that is describe in section data specification.
- Retrieve the WBCD dataset, which contains relevant data for the Breast Cancer Detection Project. The dataset comprises information extracted from fine needle aspirate (FNA) samples of breast mass.
- Data preprocessing (or Feature Engineering) to select features to be used in different Machine Learning Models
- Scale and Splitting dataset into 80% for training, 20% for validation, and testing sets.
- Build model by using default hyperparameters.
- Tuning model for optimizing and used the optimum hyperparameters to be used on testing set.

- Evaluate model and find the result by using different performance metrics such as accuracy, precision, recall, and F1-score.
- Visualize data in each step.

→ Now, I start to show the detail of models with describe important functionality of the project:

1- Dataset and Data preprocessing:

The Breast Cancer datasets is available as machine learning repository maintained by the University of California, Irvine. The dataset contains 569 samples of malignant and benign tumor cells. The first two columns in the dataset store the unique ID numbers of the samples and the corresponding diagnosis (M = malignant, B = benign), respectively. The columns 32 contain 30 real-value features that have been computed from digitized images of the cell nuclei, which can be used to build a model to predict whether a tumor is benign or malignant.

```
[6]: df.columns

[6]: Index(['diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
            'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
            'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
            'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
            'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
            'fractal_dimension_se', 'radius_worst', 'texture_worst',
            'perimeter_worst', 'area_worst', 'smoothness_worst',
            'compactness_worst', 'concavity_worst', 'concave points_worst',
            'symmetry_worst', 'fractal_dimension_worst'],
           dtype='object')
```

In the result displayed, you can see the data has 569 records, each with 31 columns. The info() method provides a concise summary of the data; from the output, it provides the type of data in each column, the number of non-null values in each column, and how much memory the data frame is using. The method get_dtype_counts() will return the number of columns of each type in a Dataframe:

```
[123]: # Review data types with "info()".
       df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 31 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   diagnosis                569 non-null    object
 1   radius_mean              569 non-null    float64
 2   texture_mean             569 non-null    float64
 3   perimeter_mean           569 non-null    float64
 4   area_mean                569 non-null    float64
 5   smoothness_mean          569 non-null    float64
 6   compactness_mean         569 non-null    float64
 7   concavity_mean           569 non-null    float64
 8   concave points_mean      569 non-null    float64
 9   symmetry_mean            569 non-null    float64
 10  fractal_dimension_mean   569 non-null    float64
 11  radius_se                569 non-null    float64
 12  texture_se               569 non-null    float64
 13  perimeter_se             569 non-null    float64
 14  area_se                  569 non-null    float64
 15  smoothness_se            569 non-null    float64
 16  compactness_se           569 non-null    float64
 17  concavity_se             569 non-null    float64
 18  concave points_se        569 non-null    float64
 19  symmetry_se              569 non-null    float64
 20  fractal_dimension_se     569 non-null    float64
 21  radius_worst             569 non-null    float64
 22  texture_worst            569 non-null    float64
 23  perimeter_worst          569 non-null    float64
 24  area_worst               569 non-null    float64
 25  smoothness_worst         569 non-null    float64
 26  compactness_worst        569 non-null    float64
 27  concavity_worst          569 non-null    float64
 28  concave points_worst     569 non-null    float64
 29  symmetry_worst           569 non-null    float64
 30  fractal_dimension_worst  569 non-null    float64
dtypes: float64(30), object(1)
memory usage: 137.9+ KB
```

After that, to check for missing variables, I use df.isnull().any() command

```
df.isnull().any()
```

```
[124]: diagnosis                 False
       radius_mean               False
       texture_mean              False
       perimeter_mean            False
       area_mean                 False
       smoothness_mean           False
       compactness_mean          False
       concavity_mean            False
       concave points_mean       False
       symmetry_mean             False
       fractal_dimension_mean    False
       radius_se                 False
       texture_se                False
       perimeter_se              False
       area_se                   False
       smoothness_se             False
       compactness_se            False
       concavity_se              False
       concave points_se         False
       symmetry_se               False
       fractal_dimension_se      False
       radius_worst              False
       texture_worst             False
       perimeter_worst           False
       area_worst                False
       smoothness_worst          False
       compactness_worst         False
```

Then I use df.diagnosis.unique() command to show the results which is M or B.

```
array(['M', 'B'], dtype=object)
```

However, from the results above, diagnosis is a categorical variable, because it represents a fixed number of possible values (i.e, Malignant, of Benign). The machine learning algorithms want numbers, and not strings, as their inputs so we need some method of coding to convert them. Now I have a good intuitive sense of the data, so, the next step involves taking a closer look at attributes and data values. And I can explore the data further easily. Also, now I can get familiar with the data, which will provide useful knowledge for data pre-processing.

In addition, we should know about Objectives of Data Exploration (EDA) is a very important step that takes place after feature engineering and acquiring data and it should be done before any modeling. This is because it is very important for a data scientist to be able to understand the nature of the data without making assumptions. The results of data exploration can be extremely useful in grasping the structure of the data, the distribution of data, and the presence of extreme values and interrelationships within the data set.

```
plt.rcParams['axes.titlesize'] = 'large'
df = pd.read_csv('data_clean_id.csv', index_col=False)
df.drop('Unnamed: 0',axis=1, inplace=True)
df.head(3)
```

[18]:

ave ean	...	radius_worst	texture_worst	perimeter_worst	area_worst	smoothness_worst	compactness_worst	concavity_worst	concave points_worst	symmetry_worst	fractal_dimension_wors
710	...	25.38	17.33	184.6	2019.0	0.1622	0.6656	0.7119	0.2654	0.4601	0.1189
017	...	24.99	23.41	158.8	1956.0	0.1238	0.1866	0.2416	0.1860	0.2750	0.0890
790	...	23.57	25.53	152.5	1709.0	0.1444	0.4245	0.4504	0.2430	0.3613	0.0875

```
df.describe()
```

[19]:

	...	radius_worst	texture_worst	perimeter_worst	area_worst	smoothness_worst	compactness_worst	concavity_worst	concave points_worst	symmetry_worst	fractal_dimension_worst
	...	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000
	...	16.269190	25.677223	107.261213	880.583128	0.132369	0.254265	0.272188	0.114606	0.290076	0.083946
	...	4.833242	6.146258	33.602542	569.356993	0.022832	0.157336	0.208624	0.065732	0.061867	0.018061
	...	7.930000	12.020000	50.410000	185.200000	0.071170	0.027290	0.000000	0.000000	0.156500	0.055040
	...	13.010000	21.080000	84.110000	515.300000	0.116600	0.147200	0.114500	0.064930	0.250400	0.071460
	...	14.970000	25.410000	97.660000	686.500000	0.131300	0.211900	0.226700	0.099930	0.282200	0.080040
	...	18.790000	29.720000	125.400000	1084.000000	0.146000	0.339100	0.382900	0.161400	0.317900	0.092080
	...	36.040000	49.540000	251.200000	4254.000000	0.222600	1.058000	1.252000	0.291000	0.663800	0.207500

```
plt.rcParams['axes.titlesize'] = 'large'
df = pd.read_csv('data_clean_id.csv', index_col=False)
df.drop('Unnamed: 0',axis=1, inplace=True)
df.head(3)
```

[18]:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	...	radius_worst	textur
0	842302	M	17.99	10.38	122.8	1001.0	0.11840	0.27760	0.3001	0.14710	...	25.38	
1	842517	M	20.57	17.77	132.9	1326.0	0.08474	0.07864	0.0869	0.07017	...	24.99	
2	84300903	M	19.69	21.25	130.0	1203.0	0.10960	0.15990	0.1974	0.12790	...	23.57	

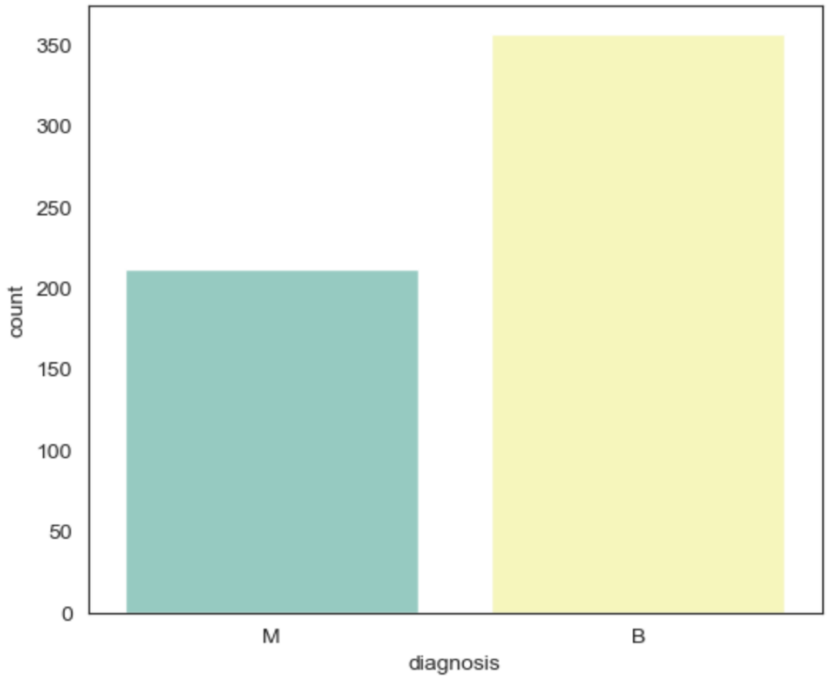
3 rows × 32 columns

```
df.describe()
```

[19]:

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	symmetry_mean	...	radi
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	...	56
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.088799	0.048919	0.181162	...	1
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.079720	0.038803	0.027414	...	
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000	0.000000	0.106000	...	
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.029560	0.020310	0.161900	...	1
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.061540	0.033500	0.179200	...	1
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400	0.130700	0.074000	0.195700	...	1
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.426800	0.201200	0.304000	...	3

For better looking to result and to get better idea about that, a plot can show that.



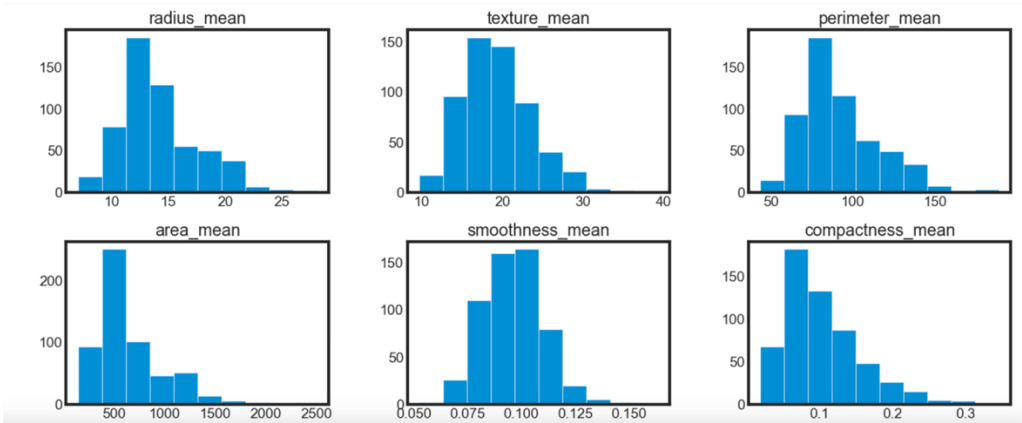

```
[13]: print(data.groupby('diagnosis').size())

diagnosis
0    357
1    212
```

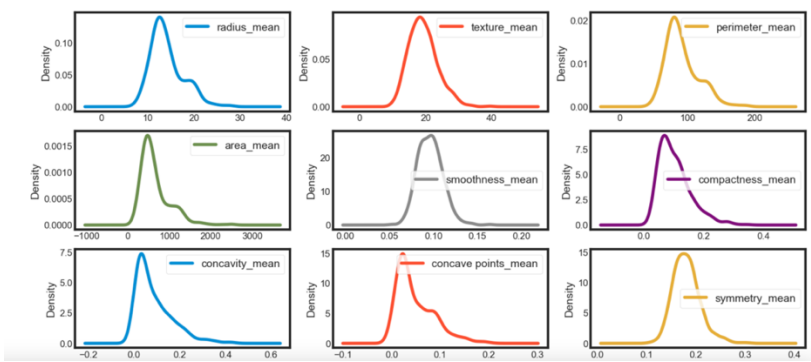
To display and visualize parts of data:

I have used different types of plots such as histograms which are commonly used to visualize numerical variables. A histogram is similar to a bar graph after the values of the variable are grouped (binned) into a finite number of intervals (bins). Since, histograms group data into bins and provide you a count of the number of observations in each bin. From the shape of the bins, you can quickly get a feeling for whether an attribute is Gaussian, skewed or even has an exponential distribution. It can also help you see possible outliers.

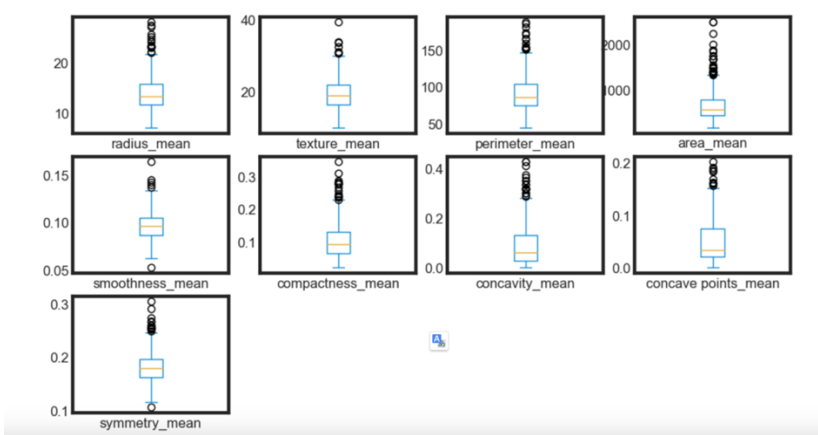
→ Histogram the _mean suffix designation:



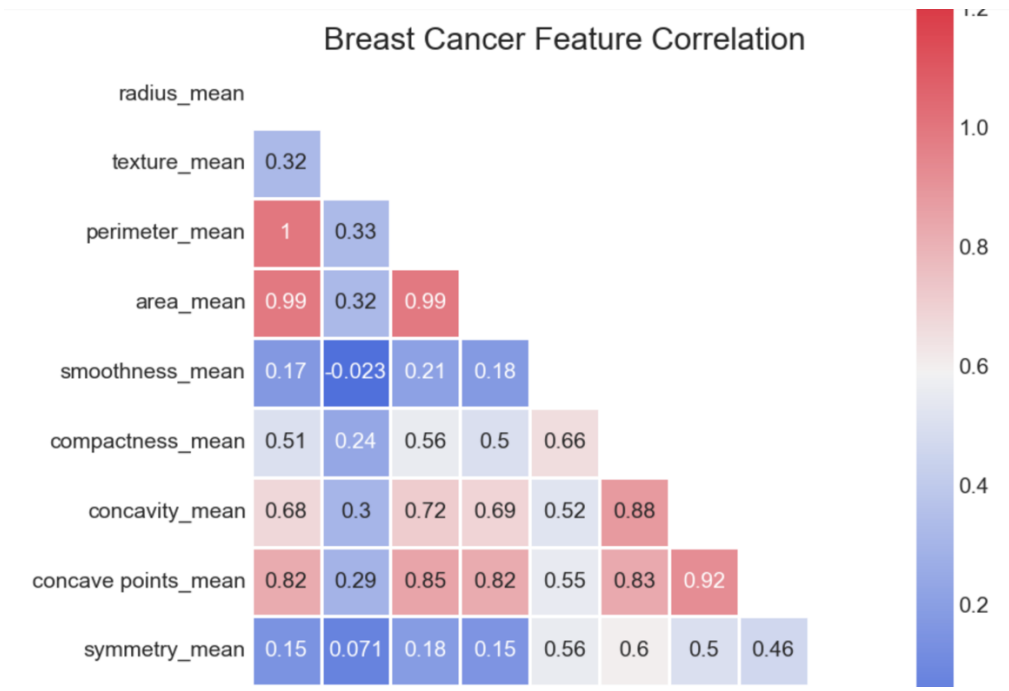
→ Density plots _mean suffix designation



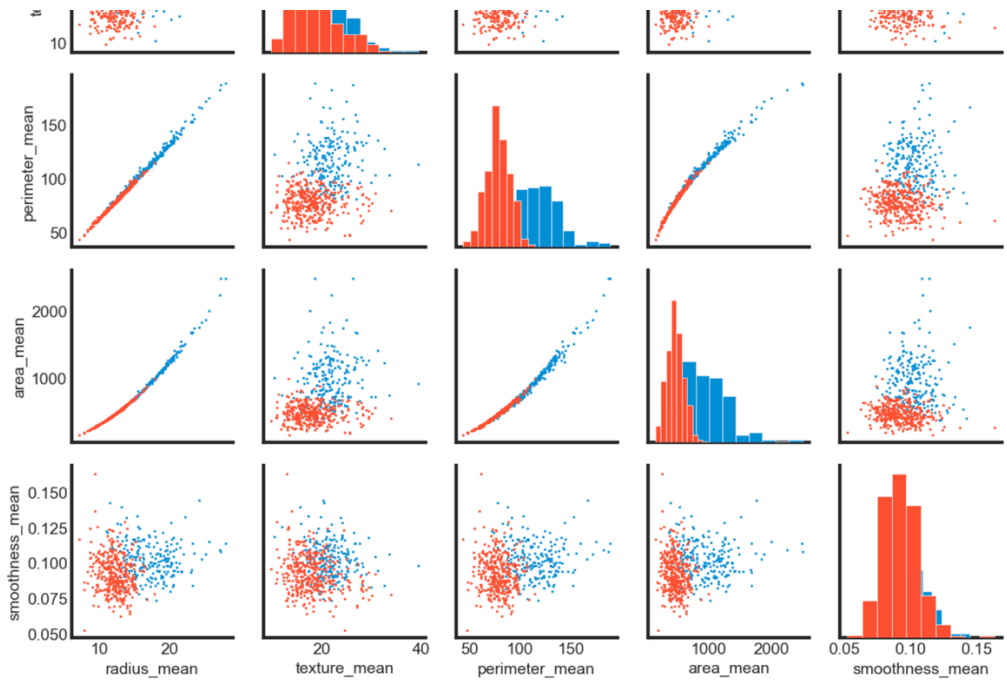
→ Visualize distribution of data via box plots:



In my code, I have utilized various Multimodal Data Visualizations, including Scatter plots and Correlation matrix, to present significant details. Scatter plots demonstrate the association between two variables, while a correlation matrix reveals the correlation coefficients among multiple variables.



Between the mean values of 1 to 0.75, we observe a strong positive relationship with the mean area of the tissue nucleus. The mean area of the tissue nucleus exhibits a strong positive correlation with the mean values of the radius and parameter. Certain parameters, such as concavity and area, concavity, and perimeter, show moderate positive correlation (with r values between 0.5 and 0.75). Similarly, we observe strong negative correlations between the fractal dimension and the mean values of radius, texture, and parameter.



In summary, the mean values of cell radius, perimeter, area, compactness, concavity, and concave points can be utilized for cancer classification. Higher values of these parameters tend to correlate with malignant tumors. However, the mean values of texture, smoothness, symmetry, and fractal dimension do not exhibit a distinct preference for one diagnosis over the other. Furthermore, none of the histograms display significant outliers that would necessitate additional data cleanup.

Preparing data is a crucial step in any data analysis problem as it enables better exposure of the problem's structure to the intended machine learning algorithms. This process involves several activities including: Assigning numerical values to categorical data, handling missing values Normalizing the features to prevent dominance of small-scale features during model fitting. In this part, I conducted an exploration of the data to gain insights into its distribution and attribute correlations. I identified certain features of interest. In this notebook, I utilize feature selection techniques to reduce the dimensionality of the high-dimensional data, along with feature extraction and transformation methods for dimensionality reduction. Besides the goal is to identify the most predictive features in the data and apply filtering techniques to enhance the predictive power of the analytics model.

Once the class labels (diagnosis) have been encoded, the malignant tumors are now denoted as class 1 (indicating the presence of cancer cells), while the benign tumors are represented as class 0 (indicating no detection of cancer cells). This representation is achieved by utilizing the transform method of the Label Encoder on two dummy variables.

```
array([[17.99, 10.38, 122.8, ..., 0.2654, 0.4601, 0.1189],
       [20.57, 17.77, 132.9, ..., 0.186, 0.275, 0.08902],
       [19.69, 21.25, 130.0, ..., 0.243, 0.3613, 0.08758],
       ...,
       [16.6, 28.08, 108.3, ..., 0.1418, 0.2218, 0.0782],
       [20.6, 29.33, 140.1, ..., 0.265, 0.4087, 0.124],
       [7.76, 24.54, 47.92, ..., 0.0, 0.2871, 0.07039]], dtype=object)
```

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y = le.fit_transform(y)
y
```

```
array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,
       1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 1, 1,
       0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1,
       0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0,
       0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1,
       1, 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0,
       0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1,
       1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1,
       0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0,
       0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 1,
       1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1,
       1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1,
```

2- Splitting dataset:

The data has been divided into training and test sets, which is a common practice for assessing the performance of machine learning algorithms. In this case, the available data will be split into a training set and a testing set, with a distribution of 80% for training and 20% for testing.

```
#Split data
Y = data['diagnosis'].values
X = data.drop('diagnosis', axis=1).values
X_train, X_test, Y_train, Y_test = train_test_split (X, Y, test_size = 0.20, random_state=21)
```

About feature standardization transforms attributes to have a mean of 0 and a standard deviation of 1, promoting better performance in machine learning algorithms. In this analysis, sklearn is used to scale and transform the data for standardized evaluation.

3- Building Models:

In this part I have used different types of algorithms which are SVM, CART, NB, and KNN.

```
#Building Models
models_list = []
models_list.append(('CART', DecisionTreeClassifier()))
models_list.append(('SVM', SVC()))
models_list.append(('NB', GaussianNB()))
models_list.append(('KNN', KNeighborsClassifier()))
```

Hyperparameters are specific settings in machine learning algorithms that are chosen before the training process. They have a significant impact on how the algorithms learn and perform. The four algorithms I used each have their own unique hyperparameters that affect their behavior during training. However, i will display and explain these models with their hyperparameters.

- Support Vector Machine (SVM) :

1- C: Regularization parameter (cost) that controls the trade-off between maximizing the margin and minimizing the classification error.
2- Kernel: SVM allows different kernel functions such as linear, polynomial, radial basis function (RBF), and sigmoid.

```
model: SVM, params: {'C': 1.0, 'break_ties': False, 'cache_size': 200, 'class_weight': None, 'coef0': 0.0, 'decisio
n_function_shape': 'ovr', 'degree': 3, 'gamma': 'scale', 'kernel': 'rbf', 'max_iter': -1, 'probability': False, 'ra
ndom_state': None, 'shrinking': True, 'tol': 0.001, 'verbose': False}, name: SVC()

SVM: 0.907681 (0.054723) (run time: 0.046383)
```

- Classification and Regression Trees (CART):

1- Maximum Depth (max_depth): Determines the maximum depth of the decision tree.
2- Minimum Samples for Split (min_samples_split): Specifies the minimum number of samples required to split an internal node.
3- Minimum Samples per Leaf (min_samples_leaf): Sets the minimum number of samples required to be at a leaf node. Similar to min_samples_split, it prevents the creation of leaves with too few samples, enhancing model generalization.

```
model: CART, params: {'ccp_alpha': 0.0, 'class_weight': None, 'criterion': 'gini', 'max_depth': None, 'max_feature
s': None, 'max_leaf_nodes': None, 'min_impurity_decrease': 0.0, 'min_samples_leaf': 1, 'min_samples_split': 2, 'min
_weight_fraction_leaf': 0.0, 'random_state': None, 'splitter': 'best'}, name: DecisionTreeClassifier()

CART: 0.918792 (0.038141) (run time: 0.091414)
```

- Naive Bayes (NB):

1- Smoothing (alpha): Also known as Laplace smoothing or add-alpha smoothing, it prevents zero probabilities in cases where a feature-value combination is unseen in the

training data. The alpha parameter controls the smoothing effect, and higher values of alpha result in stronger smoothing.

2- Distribution Assumption: Naive Bayes assumes that features are independent, given the class. Different types of NB algorithms exist based on the distribution assumption, such as Gaussian NB, Multinomial NB, and Bernoulli NB.

```
model: NB, params: {'priors': None, 'var_smoothing': 1e-09}, name: GaussianNB()

NB: 0.940773 (0.033921) (run time: 0.025908)
```

- K-Nearest Neighbors (KNN).

1- Number of Neighbors (n_neighbors): Determines the number of nearest neighbors to consider when making a prediction. Choosing an appropriate value of K is crucial, as a small K can lead to overfitting, while a large K may lead to underfitting.

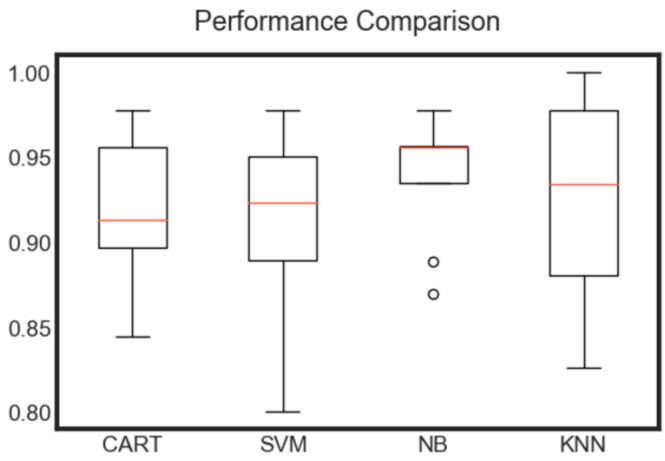
2- Distance Metric (metric): The distance metric used to measure the similarity between data points. Common metrics include Euclidean distance, Manhattan distance, and Minkowski distance. The choice of metric depends on the data and the problem at hand.

```
model: KNN, params: {'algorithm': 'auto', 'leaf_size': 30, 'metric': 'minkowski', 'metric_params': None, 'n_jobs':
None, 'n_neighbors': 5, 'p': 2, 'weights': 'uniform'}, name: KNeighborsClassifier()

KNN: 0.927729 (0.055250) (run time: 0.061643)
```

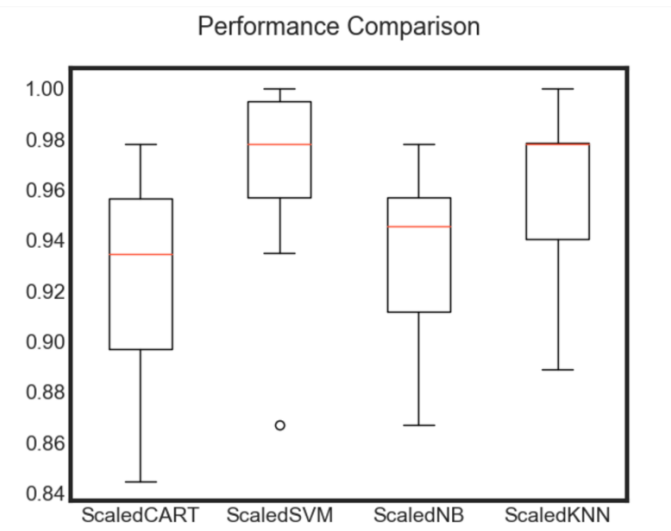
The results show a similar tight distribution for all classifiers except SVM which is encouraging, suggesting low variance. The good results for SVM are satisfactory. It is possible the varied distribution of the attributes may have an effect on the accuracy of algorithms such as SVM. In the next section we will repeat this spot-check with a standardized copy of the training dataset.

CART: 0.923285
SVM: 0.907681
NB: 0.940773
KNN: 0.927729



However, previous results were obtained using actual data without any scaling applied. And with scaling, the performances of ML algorithms have changed as shown below:

ScaledCART: 0.927633
ScaledSVM: 0.964879
ScaledNB: 0.931932
ScaledKNN: 0.958357

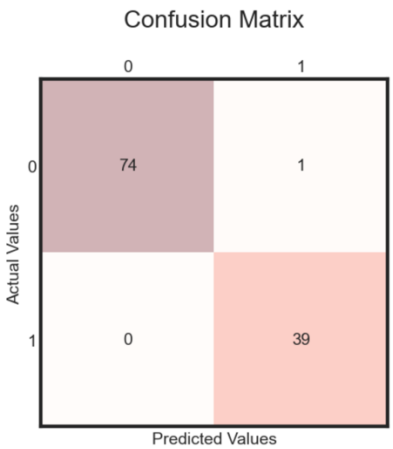


The results indicate that the data after the standardization process has made SVM the most accurate algorithm examined to date, and the results suggest further investigation into the SVM, CART, and KNN algorithms.

In short, after constructing various models, I found that SVM performed the best and had a chance for enhancement. Consequently, I fine-tuned the SVM with the most effective hyperparameters.

By training the models using the training set, I assessed the predictions on the validation set using specific metrics. The predictions were categorized into two classes: "1" representing malignant (presence of cancer cells) and "0" representing benign (absence of cancer).

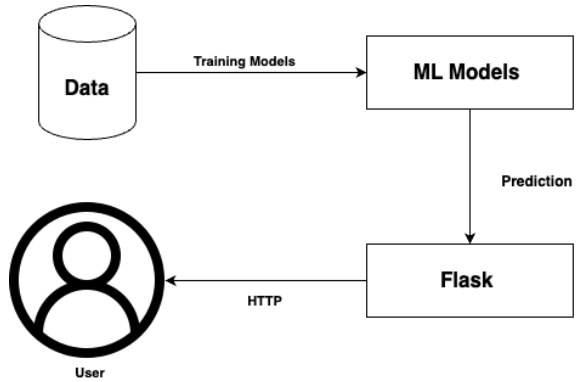
To finalize the models for breast cancer detection, I employed various evaluation metrics such as precision, F-scores, support, and recall. These metrics allowed me to assess the performance and effectiveness of the models using the Confusion Matrix technique. The Confusion Matrix provided a detailed breakdown of true positive, true negative, false positive, and false negative predictions, offering valuable insights into how well the models classified the data and their ability to detect breast cancer accurately.



	precision	recall	f1-score	support
0	1.00	0.99	0.99	75
1	0.97	1.00	0.99	39
accuracy			0.99	114
macro avg	0.99	0.99	0.99	114
weighted avg	0.99	0.99	0.99	114

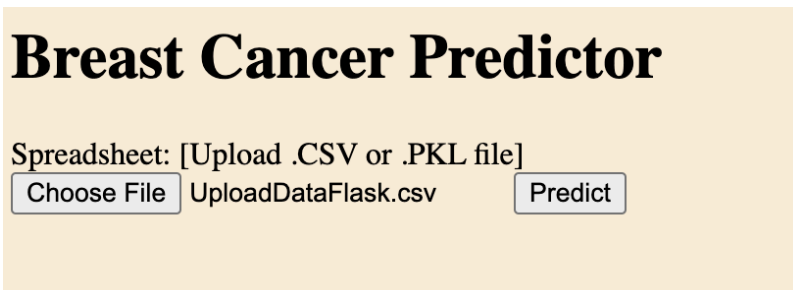
→ Nonetheless, when developing a breast cancer detection system using machine learning models, creating both a Graphical User Interface (GUI) and a Web Application is essential to display the data with different details for medical professionals. In this project, Flask, a Python web framework, will be used for building the web application. However, Graphical User Interface serves as the local interface for medical purposes to show breast cancer detection data. It provides an easy-to-use platform to upload medical files of input relevant patient data for cancer detection.

→ The figure below shows the main phases of project with using Flask for Web Application.



- Key Components of the GUI in this project:

First: Upload File is a frequently used input element in web forms, enabling users to transfer files from their local devices to a web application. In the context of breast cancer detection, users can upload medical files related to breast examinations. The GUI should be designed to accommodate files and allow users to preview the uploaded results for various analyses.



Data Information:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	10.860	21.48	68.51	360.5	0.07431	0.04227	0.00000	0.00000	0.1661	0.05948	0.3163	1.3040	2.1150	20.67	0.009579	0.011040	0.000000	0.000000	0.03004	0.00
1	14.950	18.77	97.84	689.5	0.08138	0.11670	0.09050	0.03562	0.1744	0.06493	0.4220	1.9090	3.2710	39.43	0.005790	0.048770	0.053030	0.015270	0.03356	0.00
2	20.290	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.10430	0.1809	0.05883	0.7572	0.7813	5.4380	94.44	0.011490	0.024610	0.056880	0.018850	0.01756	0.00
3	18.050	16.15	120.20	1006.0	0.10650	0.21460	0.16840	0.10800	0.2152	0.06673	0.9806	0.5505	6.3110	134.80	0.007940	0.058390	0.046580	0.020700	0.02591	0.00
4	23.510	24.27	155.10	1747.0	0.10690	0.12830	0.23080	0.14100	0.1797	0.05506	1.0090	0.9245	6.4620	164.10	0.006292	0.019710	0.035820	0.013010	0.01479	0.00
5	12.430	17.00	78.60	477.3	0.07557	0.03454	0.01342	0.01699	0.1472	0.05561	0.3778	2.2000	2.4870	31.16	0.007357	0.010790	0.009959	0.011200	0.03433	0.00
6	12.220	20.04	79.47	453.1	0.10960	0.11520	0.08175	0.02166	0.2124	0.06894	0.1811	0.7959	0.9857	12.58	0.006272	0.021980	0.039660	0.009894	0.01320	0.00
7	11.520	18.75	73.34	409.0	0.09524	0.05473	0.03036	0.02278	0.1920	0.05907	0.3249	0.9591	2.1830	23.47	0.008328	0.008722	0.013490	0.008670	0.03218	0.00
8	7.691	25.44	48.34	170.4	0.08668	0.11990	0.09252	0.01364	0.2037	0.07751	0.2196	1.4790	1.4450	11.73	0.015470	0.064570	0.092520	0.013640	0.02105	0.00
9	14.760	14.74	94.87	668.7	0.08875	0.07780	0.04608	0.03528	0.1521	0.05912	0.3428	0.3981	2.5370	29.06	0.004732	0.015060	0.018550	0.010670	0.02163	0.00

Second: prediction results: After processing the uploaded data, the GUI displays the prediction outcome, whether the results show benign or malignant characteristics.

Third: the GUI worked to the presentation of data visualizations such as plots, tables, and other graphical elements. By transforming complex data into easily understandable visuals, the GUI enhances user comprehension of the breast cancer detection system's results based on the uploaded medical files. Various types of plots, including bar charts, line graphs, pie charts, scatter plots, and histograms, can be generated to provide valuable insights into the analysis performed on the breast examination data, facilitating informed decision-making by medical professionals and users. Additionally, tables and other graphical elements complement the visualizations, further contributing to the overall interpretability and user-friendly nature of the GUI.

Predict Results

	Prediction
0	Benign
1	Benign
2	Malignant
3	Malignant
4	Malignant
5	Benign
6	Benign
7	Benign
8	Benign
9	Benign

Project Milestones

Milestone 1 Data Collection and Preprocessing

-Acquire the WBCD (Wisconsin Breast Cancer Diagnosis) dataset, which contains cytological features extracted from fine needle aspirate (FNA) samples of a breast mass.
-Verify the integrity and authenticity of the dataset to ensure its reliability for the project.
-Explore the dataset's attributes and understand their meaning and relevance to breast cancer diagnosis.
-Perform data preprocessing steps, including feature scaling.
-Clean, integrate, and preprocess the data, including feature selection, encoding, and handling class imbalance.

Milestone 2 - Feature Selection and Engineering

-Analyze the available features in the breast cancer dataset, including radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension.
-Conduct exploratory data analysis to understand the distribution, range, and potential outliers of each feature.

-Create additional features based on domain knowledge and insights from the dataset, such as ratios or combinations of existing features.
-Consider feature engineering techniques like polynomial features, interaction terms, or log transformations to capture non-linear relationships or enhance discriminatory power.

Milestone 3 - Model Selection and Training which will be 70% of the set.

-Select appropriate ML algorithms and model architectures for breast cancer detection.
-Train the models using the preprocessed data.
- Fine-tune and optimize model hyperparameters for improved performance.

Milestone 4 - Testing the Model with Evaluation and Validation which will be 30% of the set.

-Evaluate the trained models using appropriate performance metrics (e.g., accuracy, precision, recall, and F1-score).
-Validate the models on independent datasets to assess generalizability and robustness.
-Compare model performance with benchmark models or expert assessments.

Milestone 5 – Model Performance Visualization

-Generate visualizations of model performance metrics such as accuracy, precision, recall, and F1-score.
-Create line charts or bar graphs to compare the performance of different models or variations of the same model.
-Plot learning curves to analyze the convergence and overfitting of the models during training.

Milestone 6 - Performance Monitoring and Comparative Analysis.

-Continuously monitor the models' performance over time.
-Assess accuracy, efficiency, and efficacy of the models.
-Make necessary adjustments or updates to improve model efficiency and effectiveness.

Milestone 7 – GUI and Web Application

- Web Framework Selection is the process of deciding which web framework to use. In this case, Flask, a lightweight and efficient Python web framework, is chosen because it makes making web apps easy and gives you a lot of options.
- Initial Web Application, in this milestone includes setting up the project structure, defining the virtual environment, and loading Flask and other related tools.
- Data visualization is a key part of the web program. This goal is all about putting together tools like Matplotlib or Plotly to show breast cancer information in a way that is useful and easy to use.

Repository / Archive

https://github.com/Alasrar/Pro2.SoftwareDev_AI.git
https://colab.research.google.com/drive/1TbzCz9AnCAfhZE5zXKkv3-srzysxiaHW?usp=sharing

Reference Material

[1] The provided link corresponds to the "Breast Cancer Wisconsin Diagnostic" dataset available on the UCI Machine Learning Repository. This dataset contains real-valued features computed from cell nuclei characteristics and is widely used for breast cancer diagnosis and machine learning research.
[2] The provided link is a Kaggle notebook focusing on breast cancer prediction using the Breast Cancer Wisconsin (Diagnostic) dataset. It has some Important contains code and analyses which can help to do the project.
[3] The provided link leads to a Python script named "breast_cancer_analysis.py" hosted on the GitHub repository. And it has a script that involves an analysis or implementation related to breast cancer, utilizing the Python programming language.
[4] The provided link directs to a blog post on Towards Data Science titled "Building a Simple Machine Learning Model on Breast Cancer Data." The blog post discusses the process of constructing a basic machine-learning model using breast cancer data. It may cover topics such as data preprocessing, model selection, and evaluation metrics, and provide code examples or implementations.
[5] The provided link directs to a Kaggle notebook titled "SVM vs KNN in Breast Cancer." The notebook contains a comparative analysis between Support Vector Machines (SVM) and K-Nearest Neighbors (KNN) algorithms applied to breast cancer data. It may include code implementations, data exploration, model training, evaluation, and insights into the performance and differences between SVM and KNN for breast cancer classification.
[6] and [7] I have searched for scholarly articles, research papers, and sessions relating to learning more about Breast Cancer Detection Using Different Machine Learning Models

[1] https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic
[2] https://www.kaggle.com/code/buddhiniw/breast-cancer-prediction
[3] https://github.com/vishabh123/vishabh/blob/master/breast_cancer_analysis.py
[4] https://towardsdatascience.com/building-a-simple-machine-learning-model-on-breast-cancer-data-eca4b3b99fa3
[5] https://www.kaggle.com/code/ahmadalijamali/svm-vs-knn-in-breast-cancer
[6]https://ieeexplore-ieee-org.libproxy.library.unt.edu/stamp/stamp.jsp?tp=&arnumber=10010783
[7] https://ieeexplore-ieee-org.libproxy.library.unt.edu/stamp/stamp.jsp?tp=&arnumber=8487537
[8]
[9]https://medium.com/nerd-for-tech/breast-cancer-classifier-by-using-machine-learning-techniquesd17bcb034892#:~:text=Machine%20Learning%20techniques%20can%20dramatically,achieved%20using%20Machine%20Learning%20techniques.
[10] https://medium.com/codex/detecting-breast-cancer-using-machine-learning-c1357f2b62f

[11]https://medium.com/@akhilesh3091999/breast-cancer-detection-using-machine-learning-48a84ade64fd
[12]https://medium.com/@chfaisalnawaz702/breast-cancer-detection-using-machine-learning-classifier-37b544c5024c
[13]https://medium.com/analytics-vidhya/breast-cancer-detection-using-machine-learning-algorithms-1f61000508d8
[14] https://medium.com/swlh/breast-cancer-classification-using-python-e83719e5f97d
[15] https://www.frontiersin.org/journals/oncology/articles/10.3389/fonc.2023.1150840/full

Appendix:

```
#!/usr/bin/env python
# coding: utf-8

# ## Breast Cancer Prediction Project

# In[59]:


#import libraries
import numpy as np
import pandas as pd
import os
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import KFold
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC
import time
from scipy.stats import norm
import seaborn as sns # data visualization


# In[60]:


data = pd.read_csv('data.csv', index_col=False)
data.head(5)


# In[61]:
```

```
print(data.shape)

# In[62]:

data.describe()

# In[63]:

#Data visualisation and pre-processing
data['diagnosis'] = data['diagnosis'].apply(lambda x: '1' if x == 'M' else
'0')
data = data.set_index('id')

# In[64]:

df = pd.read_csv('data.csv', index_col=False)

# In[65]:

df.diagnosis.unique()

# In[66]:

diag_gr = df.groupby('diagnosis', axis=0)
pd.DataFrame(diag_gr.size(), columns=['# of observations'])

# In[67]:

sns.set_style("white")
sns.set_context({"figure.figsize": (6, 5)})
sns.countplot(x=df['diagnosis'],label='Count',palette="Set3")
# plt.savefig('output1.png')

# In[68]:

print(data.groupby('diagnosis').size())
```

```
# In[69]:


df_id_diag=df.loc[:,["id","diagnosis"]]
df_diag=df.loc[:,["diagnosis"]]
df_mean=df.iloc[:,1:11]
df_se=df.iloc[:,11:22]
df_worst=df.iloc[:,23:]
print(df_id_diag.columns)


# In[70]:


hist_mean=df_mean.hist(bins=10, figsize=(15, 10),grid=False,)


# In[71]:


hist_se=df_se.hist(bins=10, figsize=(15, 10),grid=False,)


# In[72]:


hist_worst=df_worst.hist(bins=10, figsize=(15, 10),grid=False,)


# In[73]:


plt = df_mean.plot(kind= 'density', subplots=True, layout=(4,3),
sharex=False,
                     sharey=False, fontsize=12, figsize=(15,10))


# In[74]:


#Density Plots
plt = df_se.plot(kind= 'density', subplots=True, layout=(4,3), sharex=False,
                    sharey=False, fontsize=12, figsize=(15,10))


# In[75]:


#Density Plots
plt = df_worst.plot(kind= 'kde', subplots=True, layout=(4,3), sharex=False,
sharey=False,fontsize=5, figsize=(15,10))


# In[76]:
```

```
# box and whisker plots
plt=df_mean.plot(kind= 'box' , subplots=True, layout=(4,4), sharex=False,
sharey=False,
                 fontsize=12)


# In[77]:


plt=df_worst.plot(kind= 'box' , subplots=True, layout=(4,4), sharex=False,
sharey=False,
                  fontsize=12)


# In[78]:


import matplotlib.pyplot as plt
plt.style.use('fivethirtyeight')
sns.set_style("white")
df_mean['diagnosis'] = df_mean['diagnosis'].apply(lambda x: '1' if x == 'M'
else '0')
corr = df_mean.corr()
mask = np.zeros_like(corr)
mask[np.triu_indices_from(mask)] = True
df1, ax = plt.subplots(figsize=(8, 8))
plt.title('Breast Cancer Feature Correlation')
cmap = sns.diverging_palette(260, 10, as_cmap=True)
sns.heatmap(corr, vmax=1.2, square='square', cmap=cmap, mask=mask,
            ax=ax,annot=True, fmt='.2g',linewidths=2)


# In[79]:


plt.style.use('fivethirtyeight')
sns.set_style("white")
# df = pd.read_csv('data_clean.csv', index_col=False)
g = sns.PairGrid(df[[df.columns[1],df.columns[2], df.columns[3],
                     df.columns[4], df.columns[5], df.columns[6]]],
hue='diagnosis')
g = g.map_diag(plt.hist)
g = g.map_offdiag(plt.scatter, s = 3)


# In[80]:


#Split data
Y = data['diagnosis'].values
X = data.drop('diagnosis', axis=1).values
```

```
X_train, X_test, Y_train, Y_test = train_test_split (X, Y, test_size = 0.20,
random_state=21)


# In[86]:


df = pd.read_csv('data.csv', index_col=False)
# df.drop('Unnamed: 0',axis=1, inplace=True)
array = df.columns[2]
X1 = df[df.columns[2:]].values
y1=df['diagnosis'].values


# In[92]:


df


# In[87]:


from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y1 = le.fit_transform(y1)
y1


# In[89]:


from sklearn.preprocessing import StandardScaler
scaler =StandardScaler()
Xs = scaler.fit_transform(X1)


# In[90]:


from sklearn.decomposition import PCA
pca = PCA(n_components=10)
fit = pca.fit(Xs)


# In[91]:


X_pca = pca.transform(Xs)
PCA_df = pd.DataFrame()
PCA_df['PCA_1'] = X_pca[:,0]
PCA_df['PCA_2'] = X_pca[:,1]
plt.plot(PCA_df['PCA_1'][df.diagnosis == 'M'],PCA_df['PCA_2'][df.diagnosis ==
'M'],'o', alpha = 0.7, color = 'r')
```

```
plt.plot(PCA_df['PCA_1'][df.diagnosis == 'B'],PCA_df['PCA_2'][df.diagnosis ==
'B'],'o', alpha = 0.7, color = 'b')
plt.xlabel('PCA_1')
plt.ylabel('PCA_2')
plt.legend(['Malignant','Benign'])
plt.show()


# In[28]:


#Building Models
models_list = []
models_list.append(('CART', DecisionTreeClassifier()))
models_list.append(('SVM', SVC()))
models_list.append(('NB', GaussianNB()))
models_list.append(('KNN', KNeighborsClassifier()))


# In[57]:


#Cross_Validation
num_folds = 10
results = []
names = []
param=[]
for name, model in models_list:
    kfold = KFold(n_splits=num_folds)#, random_state=123)
    start = time.time()
    cv_results = cross_val_score(model, X_train, Y_train, cv=kfold,
scoring='accuracy')
    p=model.get_params()
    param.append(model.get_params())
    end = time.time()
    results.append(cv_results)
    names.append(name)
    print(f'model: {name}, params: {p}, name: {model}\n')
    print( "%s: %f (%f) (run time: %f)" % (name, cv_results.mean(),
cv_results.std(), end-start))
    print('\n\n')


# In[30]:


fig = plt.figure(figsize=(6,4))
fig.suptitle('Performance Comparison')
ax = fig.add_subplot(111)
plt.boxplot(results)
ax.set_xticklabels(names)
plt.show()
```

```
# In[32]:


import warnings
pipelines = []
pipelines.append(('ScaledCART', Pipeline([('Scaler',
StandardScaler()),('CART',DecisionTreeClassifier())])))
pipelines.append(('ScaledSVM', Pipeline([('Scaler', StandardScaler()),('SVM',
SVC( ))])))
pipelines.append(('ScaledNB', Pipeline([('Scaler',
StandardScaler()),('NB',GaussianNB())])))
pipelines.append(('ScaledKNN', Pipeline([('Scaler',
StandardScaler()),('KNN',KNeighborsClassifier())])))
results = []
names = []
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    kfold = KFold(n_splits=num_folds)#, random_state=123)
    for name, model in pipelines:
        start = time.time()
        cv_results = cross_val_score(model, X_train, Y_train, cv=kfold,
scoring='accuracy')
        end = time.time()
        results.append(cv_results)
        names.append(name)
        print( "%s: %f (%f) (run time: %f)" % (name, cv_results.mean(),
cv_results.std(), end-start))


# In[33]:


fig = plt.figure(figsize=(6,5))
fig.suptitle('Performance Comparison')
ax = fig.add_subplot(111)
plt.boxplot(results)
ax.set_xticklabels(names)
plt.show()


# In[34]:


#Scaling means transforing Data[btween 0,1]
scaler = StandardScaler().fit(X_train)
rescaledX = scaler.transform(X_train)
#Tunning on SVM between the result of it is too high
c_values = [0.1, 0.3, 0.5, 0.7, 0.9, 1.0, 1.3, 1.5, 1.7, 2.0]
#ploy, linear, rbf, sigmid = ttpes of model kernal
kernel_values = ['linear', 'poly', 'rbf', 'sigmoid']
param_grid = dict(C=c_values, kernel=kernel_values)
model = SVC()
kfold = KFold(n_splits=num_folds)#, random_state=21)
```

```
grid = GridSearchCV(estimator=model, param_grid=param_grid,
scoring='accuracy', cv=kfold)
grid_result = grid.fit(rescaledX, Y_train)
print("Best: %f using %s" % (grid_result.best_score_,
grid_result.best_params_))
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))


# In[35]:


print(means.max(),means.size)
means.argmax()


# In[36]:


grid_result.best_params_


# In[37]:


grid_result.best_params_['kernel']


# In[ ]:





# In[38]:


# prepare the model
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    scaler = StandardScaler().fit(X_train)
X_train_scaled = scaler.transform(X_train)
model = SVC(C=2.0, kernel='rbf')
start = time.time()
model.fit(X_train_scaled, Y_train)
end = time.time()
print( "Run Time: %f" % (end-start))


# predict
```

```
# In[39]:


# estimate accuracy on test dataset
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    X_test_scaled = scaler.transform(X_test)
predictions = model.predict(X_test_scaled)


# In[40]:


print("Accuracy score %f" % accuracy_score(Y_test, predictions))
print(classification_report(Y_test, predictions))


# In[41]:


print(confusion_matrix(Y_test, predictions))


# In[42]:


cm = confusion_matrix(Y_test, predictions)
get_ipython().run_line_magic('matplotlib', 'inline')
import matplotlib.pyplot as plt
from IPython.display import Image, display
fig, ax = plt.subplots(figsize=(5, 5))
ax.matshow(cm, cmap=plt.cm.Reds, alpha=0.3)
for i in range(cm.shape[0]):
     for j in range(cm.shape[1]):
         ax.text(x=j, y=i,
                s=cm[i, j],
                va='center', ha='center')
plt.xlabel('Predicted Values', )
plt.ylabel('Actual Values')
plt.title('Confusion Matrix\n')
plt.show()
print(classification_report(Y_test, predictions ))

#!/usr/bin/env python
# coding: utf-8

# In[ ]:


import base64
import os
import pandas as pd
from io import BytesIO
from flask import Flask, request, url_for
```

```
import pickle
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle
# from PIL import Image
import numpy as np
from sklearn.preprocessing import StandardScaler

import warnings
app = Flask(__name__)

# Set Upload folder in Flask incase if the uploaded file is saved in
localhost
UPLOAD_FOLDER = os.getcwd()+'/static/files'
app.config['UPLOAD_FOLDER'] =  UPLOAD_FOLDER

def load_model(file_name='BreastCancer_SVM_trained_model_acc99.pkl'):
    model,scaler=pickle.load(open(file_name,'rb'))
    return model,scaler
model,scaler=load_model()
# img = Image.open('numberofB-M.png')
# img_data = BytesIO()
# img.save(img_data, "png")
# #Then encode the saved image file.
# encoded_img_data = base64.b64encode(img_data.getvalue()).decode('utf-8')
# image_base64 = base64.b64encode(img.de).decode('utf-8')


def load_pkl_data(file):
    data= pickle.load(open(file.filename,'rb'))
    return data
# to predict new dataset, the dataset has to have 30 features (columns).
# the function accpets 2 different files .csv and .pkl
def prediction(X_test):
    col_size=0
    # check the type of object if its list
    if(isinstance(X_test,list)):
        col_size=len(X_test[0][:])
        row_size=len(X_test[:])
    # check the type of object if its numpy
    elif(isinstance(X_test,np.ndarray)):
        col_size=X_test.shape[1]
        row_size=X_test.shape[0]
    # check the type of object if its DataFrame
    elif(isinstance(X_test,pd.DataFrame)):
        col_size=X_test.shape[1]
        row_size=X_test.shape[0]
    # check th number of columns in new data if they match with model
features
    if(col_size==model.n_features_in_):
        with warnings.catch_warnings():
            scaler = StandardScaler().fit(X_test)
            warnings.simplefilter("ignore")
            X_test_scaled = scaler.transform(X_test)
        predictions = model.predict(X_test_scaled)
```

```
        # convert prediction value from int [0,1] to [Malignant,Benign]
        diagnosis_pred = ['Malignant' if x=='1' else 'Benign' for x in
predictions]

        return predictions,diagnosis_pred,col_size,row_size
    else:
        return f'The number of columns: {col_size} are not match with model
features:',f'{model.n_features_in_}. <br>data size:{row_size}'

@app.route('/')
def home():

    return f'''
        <html>
        <head>
            <title>Breast Cancer Predictor</title>
        </head>
        <body style="background-color:AntiqueWhite;">
            <h1>Breast Cancer Predictor</h1>
            <!--
            <img src="{{image_base64}}" alt="Uploaded
Image" width="300">
            -->
            <form action="/predict" method="post" enctype="multipart/form-
data">
            <label>Spreadsheet: [Upload .CSV or .PKL file]</label><br>
                <input type="file" name="file" accept=".csv,.pkl" required>
                <input type="submit" value="Predict">
            </form>
        </body>
        </html>
    '''

@app.route('/predict', methods=['POST'])
def predict():
    found= True
    file = request.files['file']
    print(f'file:{file}\n\n')
    print(f'file.filename: {file.filename}')
    # if file.rsplit('.', 1)[1].lower()
    print(file.filename.rsplit('.', 1)[1].lower())
    # data = pd.read_csv(file)
    # print(f'data:{data}')
    # Read the image data
    # file_data = file.read()
    # print(file_data)
    if file.filename != '':
        file_path = os.path.join(app.config['UPLOAD_FOLDER'], file.filename)
        print(f'file_path: {file_path}')
        if (file.filename.rsplit('.', 1)[1].lower() =='pkl'):
            data = load_pkl_data(file)
            data = pd.DataFrame(data)
            print(data.head())
        elif (file.filename.rsplit('.', 1)[1].lower() =='csv'):
```

```
            data = pd.read_csv(file,index_col=0)
            print(data.head())
        new_pred,diagnosis_pred,col_size,row_size=prediction(data)
        diagnosis_pred1=pd.DataFrame(diagnosis_pred)
        diagnosis_pred1.columns=['Prediction']

    else:
        found=False
      # save file in local host at the file_path
        # file.save(file_path)
      # save the file
    return f'''
        <html>
        <head>
            <title>Result</title>
            <style>
            table {{
              border-collapse: collapse;
              width: 100%;
            }}
            th, td {{
              text-align: left;
              padding: 8px;
            }}
            tr:nth-child(even) {{
              background-color: #D6EEEE;
            }}
            </style>
        </head>
        <body style="background-color:AntiqueWhite">
            <h1>Data Information:</h1>
            <p>{data.to_html() if found else 'The type of file is not
supported!'}</p>
            <br>
            <!--
            <p>{new_pred}</p
            <p>{diagnosis_pred}</p>
            -->
            <h1>Predict Results</h1>

            <p>{diagnosis_pred1.to_html()}</p>
        </body>
        </html>
    '''

if __name__ == '__main__':
    app.run(debug=False,port=5005)
    # app.run()


# In[ ]:


os.getcwd()+'/numberofB-M.png'
```

```
# In[ ]:
```